Rolling Deployments

a few at a time, old and new side by side

Watch the 4-min video

In one line: a rolling deployment replaces instances a few at a time, gated by readiness, so the service keeps serving throughout; `maxSurge` and `maxUnavailable` set the pace, and because old and new versions run side by side, every change must be backward compatible.

1 Twenty servers, zero downtime

You need to ship a new version of a service running on twenty instances. Stop them all, deploy, start again? That's downtime for every user. A rolling deployment replaces instances a few at a time, so enough healthy ones are always serving. It is the default strategy for a Kubernetes Deployment.

2 How it works

Start a few new instances. Wait until they pass **readiness** (lesson 68) and receive traffic. Stop a few old ones, letting them drain. Repeat until all run the new version. Readiness is what makes it safe: a pod that reports ready too early receives traffic it can't handle, and the rollout keeps going on bad pods. `minReadySeconds` adds a soak time before a new pod counts as available.

3 maxSurge and maxUnavailable

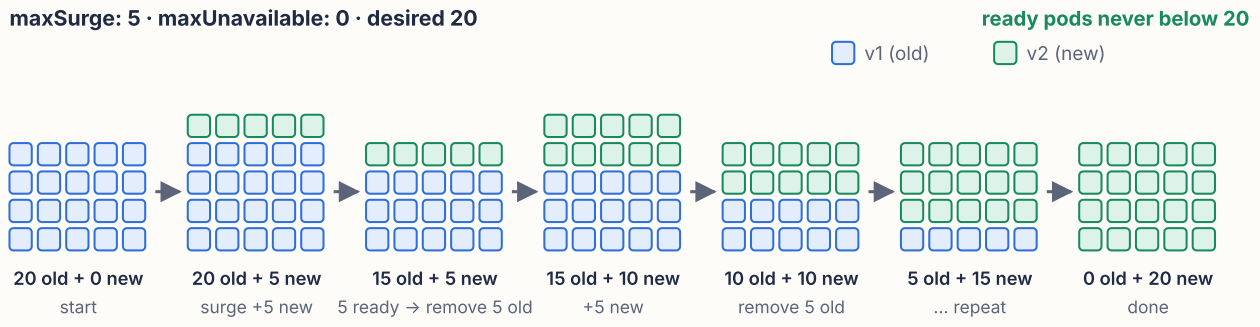


Figure 1. The video's rollout: 5 extra pods at a time, never below 20 ready. Kubernetes doesn't strictly batch; it keeps within both limits as pods become ready.

Setting	Meaning	Default	Trade-off
`maxSurge`	pods allowed **above** desired count	25% (rounded up)	higher = faster, needs spare capacity
`maxUnavailable`	pods allowed **below** desired count	25% (rounded down)	0 = never lose capacity, but needs surge > 0

Both can't be zero. `maxSurge: 0, maxUnavailable: 1` needs no extra capacity but runs one pod short; `maxSurge: 25%, maxUnavailable: 0` keeps full capacity at the cost of temporary extra pods. If new pods don't become ready within `progressDeadlineSeconds` (default 600 s), the rollout is marked as failed, but it is **not rolled back automatically**: your pipeline must act on it.

4 Version skew

During the rollout, old and new versions run side by side, sometimes for many minutes, and one user's requests may hit both. If the new version writes data the old one can't read, sends an API field the old one rejects, or publishes a message format old consumers don't understand, requests fail *in the middle* of the rollout, and again during a rollback.

5 Backward compatibility: expand and contract



Figure 2. Expand and contract (parallel change) spreads a breaking schema change over several releases so every intermediate state is safe.

Add new fields; don't remove or rename old ones in the same release. Readers tolerate unknown fields and missing new ones. For databases, **expand** (add the new column), deploy code that handles both, migrate data, then **contract** (remove the old column) only after no running version needs it. The same rule applies to queues and events.

TRADE-OFFS OF ROLLING DEPLOYMENTS

- **Cheap:** no second environment; only the surge pods are extra.
- **Slow rollback:** undo is just another rolling update, minutes not seconds.
- **Partial exposure:** a bad version serves a growing share of users while it rolls out.
- **Needs automation:** watch error rate and latency, pause or undo on regression; for risky changes use canary (lesson 75) or blue-green (lesson 74).

WORKED EXAMPLE: 20 PODS, MAXSURGE 5, MAXUNAVAILABLE 0

The orders Deployment runs **20 pods** with `strategy.rollingUpdate: {maxSurge: 5, maxUnavailable: 0}`. Kubernetes starts 5 v2 pods, waits for readiness, removes 5 v1 pods, and continues until all 20 are v2 (Figure 1): about four rounds. Capacity never drops below 20. The v2 code reads both the old and new order formats, so no request breaks while versions mix. When error rates rise, `kubectl rollout undo deployment/orders` rolls back the same way.

6 Key terms

Rolling update: replace instances gradually, a few at a time

maxUnavailable: pods allowed missing below desired

Expand and contract: add, migrate, then remove in separate releases

maxSurge: extra pods allowed above desired during rollout

Version skew: old and new versions running together

rollout undo: roll back to the previous ReplicaSet

7 Interview questions

Q1. What compatibility issues arise during a rolling deploy? BASIC
Old and new versions run at the same time, so APIs, message formats and database schemas must stay backward and forward compatible: additive changes and expand-and-contract migrations.

Q2. What do maxSurge and maxUnavailable control? BASIC
How fast the rollout goes (extra pods allowed above desired) and how much capacity it may lose (pods allowed below desired).

Q3. You must rename a database column used by the service. Plan it. SCENARIO
Release 1: add the new column, write both, read either. Backfill. Release 2: read only the new one. Release 3, once no old version runs: drop the old column.

Q4. A cluster has no spare capacity. Which settings? SCENARIO
maxSurge: 0, maxUnavailable: 1 (or a small %): no extra pods, at the cost of running slightly under capacity during the rollout.

Q5. Does Kubernetes roll back automatically if new pods fail readiness? TRICKY
No: the rollout stalls and is marked failed after progressDeadlineSeconds; old pods keep serving. Rollback is a separate action (rollout undo or your pipeline).

Q6. Why is rollback during a rolling deploy also risky? TRICKY
Rolling back creates version skew again, in reverse: if v2 already wrote data or messages v1 can't read, the rollback breaks requests too.

Next: SD-N-016 · Blue-Green Deployments backendroadmap.com